Project Proposal: Frequent-Item Detection in Data Streams (T9)

Kicki Rohde-Aagaard - kiroh22@student.sdu.dk
Mathias Nick Ziadmanesh - mazia23@student.sdu.dk
Michael Hangaard Hansen - michh16@student.sdu.dk
DM8106 Mining Massive Datasets
University of Southern Denmark

Abstract

We plan to benchmark compact one-pass summaries for frequency estimation and frequent-item detection in massive streams, comparing accuracy, time and memory against exact counting.

1 Problem Statement

Streams of purchases, page requests or events can contain far more distinct items than fit in memory, so exact counting is often infeasible. We want to study how compact one-pass summaries estimate item frequencies and discover frequent items, and how they trade accuracy against time and memory.

Tasks.

(1) **Frequency estimation (primary):** estimate $f(x)$ for any queried item x.
(2) **Top-k / heavy hitters (secondary):** report the k most frequent items, or all items above the threshold $f(x) \geq \phi N$.
(3) *(Optional)* **Drift:** popularity changes over fixed time windows.

2 Research Questions

We address the four questions in the topic description (T9):

- **RQ1:** How can frequencies and frequent items be estimated without storing the stream?
- **RQ2:** How do the methods compare in accuracy, top-k quality, time and memory?
- **RQ3:** How do skew, number of distinct items and popularity drift affect them?
- **RQ4:** What are the bottlenecks and quality-scalability tradeoffs as stream size N and memory budget vary?

2.1 Refined experimental subquestions

3 Project Objectives

(1) Benchmark at least four streaming summaries against an exact baseline on the chosen stream/streams.
(2) Explain each method's idea, parameters, guarantees, complexity and bottlenecks.
(3) Measure accuracy, top-k/heavy-hitter quality, throughput, and memory at equal budgets.
(4) Quantify the effect of skew, distinct-item count, and stream length.
(5) Deliver a reproducible repository and a 4–6 page report.

Table 1: Subquestions and planned evidence.

RQ	Subquestions	Evidence
RQ1	Which summary sizes (bytes) suffice for a given error ε? Do measured errors respect the theoretical guarantees (N/k for counters, εN w.p. $1-\delta$ for sketches)?	Error vs. memory sweep; measured maximum error vs. bounds
RQ2	At equal memory, which method has the lowest frequency error, best top-k recall, and highest throughput?	Fixed-budget comparison on all datasets
RQ3	How do skew, number of distinct items and popularity drift change the error and top-k quality of each method?	Synthetic and/or real streams with varied skew and distinct-item count; drift over time windows (optional)
RQ4	How do update time, memory and quality scale with N and budget? Which part dominates (hashing, hash-map lookup, cache misses, Python overhead)?	Budget sweep, profiling

4 Libraries & Candidate Algorithms

4.1 Potential libraries

At least four are benchmarked, from different families. The choice of library is itself part of the investigation: early in the project, we will evaluate which libraries are most suitable for processing and analyzing the data, considering coverage of the algorithms, timing and memory measurement, scalability, reproducibility, and quality of documentation. The candidates so far are as follows:

- **Apache Spark (PySpark):** built in exact counting, sampling, `DataFrame.stat.freqItems` (Misra–Gries-style) and `DataFrame.stat.countMinSketch`.
- **Streaming-sketch libraries (e.g., Apache DataSketches, `pyprobables`):** Implementations of frequent-items and Count-Min sketches.
- **Own thin wrappers or implementations** where no library covers a method, validated against a reference.

We will probably use a hybrid: a sketch library for the algorithm comparison, and Spark for data preparation, exact ground truth and a distributed scalability experiment. The final choice will be justified in the report.
Lastly, we will also need a profiling library to measure time and memory performance, e.g. **memory-profiler**, **cProfile**, or **Scalene**.

4.2 Algorithm principles and parameters

Exact counter (baseline): one counter per distinct item. No parameters, but memory grows with the number of distinct items.

Reservoir sampling: keeps a random sample of s items and scales sample counts by N/s. Parameter: sample size s. Poor for rare items.

Misra–Gries: keeps $k-1$ counters; when a new item does not fit, all counters are decreased by one. Underestimates by at most N/k. Parameter: k.

Space-Saving: keeps k counters; a new item replaces the smallest counter and inherits its count. Overestimates by at most N/k. Parameter: k.

Count-Min Sketch: a $d \times w$ table of counters with one hash function per row; the estimate is the smallest of the d counters for the item. Never underestimates. Parameters: width w and depth d.

Count Sketch: like Count-Min, but each update adds +1 or −1 (chosen by a hash) and the estimate is the median over rows. Can over- or underestimate, but is unbiased. Parameters: w, d.

Sketches answer queries for any item but need a separate candidate list for top-k.

5 Dataset

Chess games, high Elo
(`RobertZ2011/standard-chess-games-high-elo`, Hugging Face[1]). Rated standard Lichess games, one row per game (Table 2).

Table 2: Dataset columns.

Column	Type	Values / range
`Event`, `Site`	string	text
`White`, `Black`	string	player names
`Result`	class	4 values
`WhiteTitle`, `BlackTitle`	class	12 values
`WhiteElo`, `BlackElo`	int16	2000–3200
`WhiteRatingDiff`	int16	−237 to 554
`BlackRatingDiff`	int16	−262 to 518
`UTCDate`	date32	date
`UTCTime`	time32[ms]	time
`ECO`	class	494 values
`Opening`	string	opening name
`Termination`	class	5 values
`TimeControl`	class	323 values
`movetext`	string	moves (PGN)

- **Size:** about 3.41 million games (rows), 7.03 GB, dominated by the `movetext` column.
- **Streams (items):** games are read in chronological order (`UTCDate`, `UTCTime`) and the item can be chosen per experiment, giving streams with different numbers of distinct items and skew: low-cardinality categorical fields (`ECO`, `Opening`, `TimeControl`, `Termination`), high-cardinality player names (`White`/`Black`), and, optionally, moves or move prefixes extracted from `movetext`.
- **Ground truth:** exact counts by a full pass over the data.
- **Preprocessing:** drop unused columns, handle missing values (e.g. titles), and parse `movetext` if we end up wanting to do streams based on moves.
- **Scalability:** prefixes of the stream of increasing length N, and optionally sampling or concatenation to reach other sizes.
- **Why:** real, naturally skewed data with a timestamp that supports the optional drift study (e.g. opening popularity over time), and with several item types in one source, so RQ3 can be studied without changing dataset.

[1] https://huggingface.co/datasets/RobertZ2011/standard-chess-games-high-elo

6 Evaluation Plan

- **Baseline:** exact counts on the data stream.
- **Quality:**
  - Frequency estimation: mean absolute and relative error, and maximum error, over top items and over random items.
  - Top-k and heavy hitters: precision, recall and rank correlation.
- **Cost:** wall-clock update and query time, throughput (items per second), and peak memory.
- **Scalability:** vary stream size N over several scales, memory budget, skew and number of distinct items. Plots showing runtime, memory and quality vs. N.
- **Reproducibility:** fixed seeds, 5–10 repeats, documented hardware and library versions.

7 Expected Bottlenecks

- Exact counting: memory grows with the number of distinct items.
- Sketches: hashing cost per update and cache misses; a heap or candidate set is needed for top-k.
- Counter-based methods: structure maintenance and lookup cost; error grows with flat distributions.
- Sampling: high variance for rare items.